

Self Evaluation

Team Can't Recall: William Berthouex, Noah Sleeman CSC 575 — Final Project

1. How the project fulfills each evaluation criterion

Code submission. A complete, runnable prototype is included. The search engine runs with `python -m explorer index <dir>` followed by `python -m explorer search "<query>" [--prf] [--boost]`. The corpus downloader (`python -m corpus.build_corpus`) and the evaluation script (`python -m evaluation.evaluate`) is also runnable.

Code documentation The `README.md` gives setup, a quick start, every command and flag, the dataset instructions, an architecture diagram, the results, and the project layout. Every module carries a section explaining its role and the IIR technique it implements. Inline comments also provide additional context to the specific methods.

Usage of basic IIR techniques Tokenization, stopword removal, Porter stemming, an inverted index, TF-IDF (l_{tc}) weighting, cosine similarity ranking, and the standard evaluation metrics (P@k, MAP, NDCG@k). The index, weighting, cosine and metrics are build from scratch; stemming uses NLTK's Porter stemmer, but other than that we use our own L2 normalizing vectors and inverted index, TF-IDF and cosine similarity for scoring.

Usage of advanced IIR techniques Two advanced components on top of the baseline, both written by us: (1) a query intent classifier driving a query conditioned, file-type boost; and (2) Rocchio pseudo-relevance feedback. Both are independently toggleable, and a 2×2 ablation quantifies each one's contribution to the performance of the IR model. Both ideas are built from scratch and aim to contribute to our project objectives and sparked via our individual research and class concepts learned in the course.

Model complexity The retrieval algorithms are our own code that were repurposed from what we learned durring the course: the inverted index, TF-IDF weighting, cosine ranking, the intent classifier, the file-type boost, the Rocchio feedback loop, and all evaluation metrics. We use established libraries only for routine, benign IR plumbing/preprocessing: `nltk` for Porter stemming and `BeautifulSoup/pypdf` for HTML/PDF text extraction (`typer/rich` for the CLI, `matplotlib` for figures). In other words, the parts that *are* information retrieval are implemented from scratch; the parts that are file/format handling reuse standard tools that are out of scope of this course.

Usage of good data. A heterogeneous corpus of 650 indexed documents (67,610 unique terms) across six file categories (code, text, notebook, html, pdf), collected from public sources (Gutenberg, Wikipedia, arXiv, GitHub). far larger than the assignment examples but runnable on one laptop. A manifest records each document's provenance, which doubles as the relevance labels for evaluation.

Good experiments and evaluation. We report P@10, MAP, and NDCG@10 over 28+ queries; run the full 2×2 ablation; and grid search the hyperparameters (`prf_k`, `beta`, `gamma`), reporting both the best configuration and one-dimensional sweeps showing each parameter's effect. We include qualitative success and failure cases, including an honest PRF query noise failure (q20).

Project objectives We dropped the manually compiled corpus, the GUI stretch goal, and live user feedback evaluation, and instead delivered: a public multi-file type corpus; two concrete advanced features the instructor asked for (file-type relevance via query intent boosting, and reproducible feedback via Rocchio PRF); and a 2×2 ablation evaluated with P@10/MAP/NDCG@10. The objective that "different file types may be more or less relevant depending on the query" attempted through the intent classifier + boost and measured directly.

2. Team member contributions

Noah Sleeman — advanced features (intent classifier, file type boost, Rocchio PRF); retrieval core (inverted index, TF-IDF, cosine); corpus builder (sources, manifest).

William Berthouex — file-explorer CLI foundation and command wiring, preprocess/index/search; evaluation design (queries, qrels methodology, metrics); documentation.

Shared — the report and presentation.

3. Evidence pointers

- Basic IIR: `explorer/ir/preprocess.py`, `explorer/ir/index.py`, `explorer/ir/rank.py` (cosineScores).
- Advanced IIR: `explorer/ir/intent.py`, `explorer/ir/rank.py` (applyBoost, rocchioExpand), `explorer/ir/search.py`.
- Experiments: `evaluation/` and `data/eval_results.json`; figures in `report/figures/`.

4. Use of AI

The one-time data-collection script (`corpus/build_corpus.py`) was written with the help of an AI assistant; it downloads and labels public documents and is not part of the IR system itself. We disclose this for transparency. The retrieval components that the project is graded on ie the inverted index, TF-IDF weighting, cosine ranking, intent classifier, file-type boost, Rocchio feedback and evaluation metrics. The corpus builder does nothing but compile a corpus for us to use our IR project on.